Experiment # 10

Implementation of Searching Algorithms in C++ programming language

Objectives:

The objective of this lab is to understand the basic concept of different searching techniques

- Linear Search.
- Binary Search.
- Jump Search.
- Interpolation Search.

Software Tools:

- Microsoft Visual Studio/ Code Blocks

Theory:

The searching algorithms are used to search or find one or more than one element from a dataset. These types of algorithms are used to find elements from a specific data structure. Searching may be sequential or not. If the data in the dataset are random, then we need to use sequential searching. Otherwise we can use other different techniques to reduce the complexity.

Search algorithms form an important part of many programs. Some searches involve looking for an entry in a database, such as looking up your record in the IRS database. Other search algorithms trawl through a virtual space, such as those hunting for the best chess moves. Although programmers can choose from numerous search types, they select the algorithm that best matches the size and structure of the database to provide a user-friendly experience.

The general searching problem can be described as follows: Locate an element x in a list of distinct elements a1,a2,...an or determine that it is not in the list. The solution to this search problem is the location of the term in the list that equals x and is 0 if x is not in the list.

Linear Search:

The linear search is the algorithm of choice for short lists, because it's simple and requires minimal code to implement. The linear search algorithm looks at the first list item to see whether you are searching for it and, if so, you are finished. If not, it looks at the next item and on through

each entry in the list.

Linear search is the basic search algorithm used in data structures. It is also called as sequential search. Linear search is used to find a particular element in an array. It is not compulsory to arrange an array in any order (Ascending or Descending) as in the case of binary search.

Example:

```
#include<iostream>
using namespace std;
// A function to perform linear search.
// this method makes a linear search.
int find(int *intarray, int n, int item) {
   int i;
   int comparisons = 0;
   // navigate through all items
   for(i = 0;i<n;i++) {
      // count the comparisons made comparisons++;
      // if item found, break the loop
      if(item == intarray[i]) {
         cout<<"element found at:"<<i<<endl;
         break;
      }
      // If index reaches to the end then the item is not there.
      if(i == n-1) {
         cout<<"\nThe element not found.";
         return -1;
      }
   }
   printf("Total comparisons made: %d", comparisons);
   // Shift each element before matched item.
```

```
   for(int j = i; j > 0; j--)
      intarray[j] = intarray[j-1];
      // Put the recently searched item at the beginning of the data
array.
      intarray[0] = item;
   return 0;
}
int main() {
   int
intarray[20]={1,2,3,4,6,7,9,11,12,14,15,16,26,19,33,34,43,45,55,66};
   int i,n;
   char ch;
   // print initial data array.
   cout<<"\nThe array is: ";
   for(i = 0; i < 20;i++)
      cout<<intarray[i]<<" ";
   up:
   cout<<"\nEnter the Element to be searched: ";
   cin>>n;
   // Print the updated data array.
   if(find(intarray,20, n) != -1) {
      cout<<"\nThe array after searching is: ";
      for(i = 0; i <20;i++)
         cout<<intarray[i]<<" ";
   }
   cout<<"\n\nWant to search more.......yes/no(y/n)?";
   cin>>ch;
   if(ch == 'y' || ch == 'Y')
      goto up;
   return 0;
```

```
}
```

Output:

```
The array is: 1 2 3 4 6 7 9 11 12 14 15 16 26 19 33 34 43 45 55 66
Enter the Element to be searched: 26
element found at:12
Total comparisons made: 13
The array after searching is: 26 1 2 3 4 6 7 9 11 12 14 15 16 19 33
34 43 45 55 66

Want to search more.......yes/no(y/n)?y

Enter the Element to be searched: 0

The element not found.

Want to search more.......yes/no(y/n)?n
```

Binary Search:

Binary Search is one of the most fundamental and useful algorithms in Computer Science. It describes the process of searching for a specific value in an ordered collection.

Binary search is a popular algorithm for large databases with records ordered by numerical key. Example candidates include the IRS database keyed by social security number and the DMV records keyed by driver's license numbers. The algorithm starts at the middle of the database — if your target number is greater than the middle number, the search will continue with the upper half of the database. If your target number is smaller than the middle number, the search will continue with the lower half of the database. It keeps repeating this process, cutting the database in half each time until it finds the record. This search is more complicated than the linear search but for large databases it's much faster than a linear search.

This algorithm can be used when the list has terms occurring in order of increasing size

Binary Search is generally composed of 3 main sections:

Pre-processing — Sort if collection is unsorted.

Binary Search — Using a loop or recursion to divide search space in half after each comparison.

Post-processing — Determine viable candidates in the remaining space.

In its simplest form, Binary Search operates on a contiguous sequence with a specified left and right index. This is called the Search Space. Binary Search maintains the left, right, and middle indices of the search space and compares the search target or applies the search condition to the middle value of the collection; if the condition is unsatisfied or values unequal, the half in which the target cannot lie is eliminated and the search continues on the remaining half until it is successful. If the search ends with an empty half, the condition cannot be fulfilled and target is not found.

```
// C++ program to implement recursive Binary Search
#include <bits/stdc++.h>
using namespace std;

// A recursive binary search function. It returns
// location of x in given array arr[l..r] is present,
// otherwise -1
int binarySearch(int arr[], int l, int r, int x)
{
    if (r >= l) {
        int mid = l + (r - l) / 2;

        // If the element is present at the middle
        // itself
        if (arr[mid] == x)
            return mid;

        // If element is smaller than mid, then
        // it can only be present in left subarray
        if (arr[mid] > x)
            return binarySearch(arr, l, mid - 1, x);

        // Else the element can only be present
        // in right subarray
        return binarySearch(arr, mid + 1, r, x);
    }

    // We reach here when element is not
    // present in array
    return -1;
}

int main(void)
{
    int arr[] = { 2, 3, 4, 10, 40 };
    int x = 10;
    int n = sizeof(arr) / sizeof(arr[0]);
    int result = binarySearch(arr, 0, n - 1, x);
    (result == -1) ? cout << "Element is not present in array"
```

```
                    : cout << "Element is present at index " << result;
    return 0;
}
```

Jump Search:

Like Binary Search, Jump Search is a searching algorithm for sorted arrays. The basic idea is to check fewer elements (than linear search) by jumping ahead by fixed steps or skipping some elements in place of searching all elements.

For example, suppose we have an array arr[] of size n and block (to be jumped) size m. Then we search at the indexes arr[0], arr[m], arr[2m]…..arr[km] and so on. Once we find the interval (arr[km] < x < arr[(k+1)m]), we perform a linear search operation from the index km to find the element x.

Let's consider the following array: (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610). Length of the array is 16. Jump search will find the value of 55 with the following steps assuming that the block size to be jumped is 4.

STEP 1: Jump from index 0 to index 4;

STEP 2: Jump from index 4 to index 8;

STEP 3: Jump from index 8 to index 12;

STEP 4: Since the element at index 12 is greater than 55 we will jump back a step to come to index 8.

STEP 5: Perform linear search from index 8 to get the element 55.

Interpolation Search:

The Interpolation Search is an improvement over Binary Search for instances, where the values in a sorted array are uniformly distributed. Binary Search always goes to the middle element to check. On the other hand, interpolation search may go to different locations according to the value of the key being searched. For example, if the value of the key is closer to the last element,

interpolation search is likely to start search toward the end side.

To find the position to be searched, it uses following formula.

```
// The idea of formula is to return higher value of pos
// when element to be searched is closer to arr[hi]. And
// smaller value when closer to arr[lo]
pos = lo + [ (x-arr[lo])*(hi-lo) / (arr[hi]-arr[Lo]) ]

arr[] ==> Array where elements need to be searched
x     ==> Element to be searched
lo    ==> Starting index in arr[]
hi    ==> Ending index in arr[]
```

Algorithm

Rest of the Interpolation algorithm is the same except the above partition logic.

Step1: In a loop, calculate the value of "pos" using the probe position formula.

Step2: If it is a match, return the index of the item, and exit.

Step3: If the item is less than arr[pos], calculate the probe position of the left sub-array. Otherwise calculate the same in the right sub-array.

Step4: Repeat until a match is found or the sub-array reduces to zero.

Lab Task

1. **Write** C++ codes for the Jump or Interpolation search.
2. **Write** a C++ program to implement binary search on an integer in a single linked list.